

1. 题目深度解构

题意重述

给定一个 $n \times m$ 网格，蛇初始长度为 1，位于 (rs, cs) 。

接下来会依次出现 $nm - 1$ 个苹果。每次苹果出现时，它一定不在当前蛇身上。你需要输出一段由 U/D/L/R 组成的移动序列，使得：

- 蛇头最后一步正好到达苹果；
- 在到达苹果之前不能提前进入苹果格；
- 移动过程中不能出界，不能撞到自己；
- 每次输出的字符串长度不超过 nm 。

吃到苹果时，蛇尾不移动，蛇长加一。普通移动时，蛇尾正常前进。

约束分析

$$2 \leq n, m \leq 50, \quad nm \leq 100$$

网格总格数很小，但这是交互式构造题。不能依赖复杂搜索，因为搜索可能把自己走进死局。更稳的方向是构造一个全局安全策略。

题目类型判定

构造题 / 贪吃蛇安全路径构造。

核心思想是：

让蛇头始终沿着一条固定的“环形路线”前进。这样蛇身永远是这条路线上的一段连续区间，不会撞到自己。

官方题解的核心也是让蛇沿环线走；当行列均为奇数时，用一个带分支的环线构造处理。

2. 思维路径推导

朴素想法为什么不稳

最自然的想法是：每次用 BFS 找一条到苹果的最短路。

但这是危险的。短期最短路可能会把蛇带进一个狭窄区域，使得后续苹果无法到达。贪吃蛇问题的本质不是“当前能不能到苹果”，而是“吃完之后是否还能继续保持可行性”。

所以我们需要一个不会把自己困住的全局不变量。

关键观察：固定环线

如果网格里存在一条经过所有格子的哈密顿环，那么策略非常简单：

- 把所有格子排成一个环。
- 蛇头永远沿环的下一个格子走。
- 每次苹果出现后，沿环走到苹果即可。

为什么安全？

因为蛇身始终是环上的一段连续区间。苹果不在蛇身上，所以从蛇头沿环往前走，第一次遇到苹果之前不会进入蛇身内部。最坏走一圈不到：

$$nm - 1$$

步，满足题目要求。

偶数情况

如果 n 或 m 至少有一个是偶数，矩形网格存在哈密顿环。

例如 n 为偶数时，可以这样走：

第一行从左到右；
之后在第 2 到第 m 列之间蛇形走完整个网格；
最后沿第 1 列回到起点。

如果 m 为偶数，就把行列转置后使用同样构造。

奇数 × 奇数情况的障碍

如果 n 和 m 都是奇数，那么格子总数是奇数。网格图是二分图，而二分图中的环长度一定是偶数，所以不可能存在经过所有格子的哈密顿环。

这就是本题真正的构造难点。

奇数 × 奇数的 Aha moment：带分支的伪环

虽然没有真正的哈密顿环，但可以构造一个“几乎是环”的路线。

使用 1-index 坐标描述，特殊格子如下：

S = (2, 1)
A = (1, 1)
B = (2, 2)
C = (1, 2)

我们构造一条经过除 B = (2, 2) 外所有格子的环：

S -> A -> C -> ... -> S

同时额外允许一条替代分支：

S -> B -> C

也就是说，每次蛇头走到 S = (2,1) 时，可以选择：

S -> A -> C

或者：

```
S -> B -> C
```

这两个分支长度相同，都是两步。每绕一圈会经过 A 或 B 中的一个。若苹果在 A，就选择 A；若苹果在 B，就选择 B；否则选择当前不会撞到蛇身的那个分支。

为什么一定安全？

在吃最后一个苹果前，蛇长最多是：

$$nm - 1$$

而这条伪环的周期也是：

$$nm - 1$$

同一个格子再次出现时，旧的那个位置至多是蛇尾；普通移动允许进入蛇尾刚刚离开的格子。若最后一步是吃苹果，则题目保证苹果不在蛇身上，所以也不会撞。

这就补上了奇数 × 奇数无法构造哈密顿环的问题。

3. Trick 与陷阱

Trick 点拨

核心 Trick 是“把贪吃蛇转化为固定路线上的队列”。

蛇身不再被看成二维复杂形状，而是固定路线上的一段连续区间。每次移动相当于这个区间整体向前滑动；吃苹果时，区间长度加一。

易错点

1. 交互题中，每次输出后要 flush。非交互改版中不 flush 也行，但保留 flush 更稳。
2. 普通移动允许进入当前蛇尾，因为蛇尾会同时离开；吃苹果的最后一步不允许进入蛇尾。不过苹果位置保证不在蛇身上。
3. 奇数 × 奇数时不能强行构造哈密顿环，必须使用分支伪环。
4. 坐标输入是 1-index，代码内部最好转成 0-index。
5. 样例输出不唯一，只要移动合法即可。

4. 代码实现 C++23

```
#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

struct P {
```

```

    int x, y;
};

vector<P> evenCycle(int n, int m) {
    vector<P> a;

    if (n % 2 == 0) {
        a.push_back({0, 0});

        for (int j = 1; j < m; j++) {
            a.push_back({0, j});
        }

        for (int i = 1; i < n; i++) {
            if (i % 2 == 1) {
                for (int j = m - 1; j >= 1; j--) {
                    a.push_back({i, j});
                }
            } else {
                for (int j = 1; j < m; j++) {
                    a.push_back({i, j});
                }
            }
        }

        for (int i = n - 1; i >= 1; i--) {
            a.push_back({i, 0});
        }
    } else {
        auto b = evenCycle(m, n);
        for (auto [x, y] : b) {
            a.push_back({y, x});
        }
    }

    return a;
}

vector<P> oddBase(int n, int m) {
    vector<P> a;

    a.push_back({1, 0});
    a.push_back({0, 0});

    for (int j = 1; j < m; j++) {
        a.push_back({0, j});
    }

    for (int i = 1; i < n - 2; i++) {
        if (i % 2 == 1) {
            for (int j = m - 1; j >= 2; j--) {
                a.push_back({i, j});
            }
        }
    }
}

```

```

        } else {
            for (int j = 2; j < m; j++) {
                a.push_back({i, j});
            }
        }
    }

    for (int t = 0, j = m - 1; j >= 2; j--, t++) {
        if (t % 2 == 0) {
            a.push_back({n - 2, j});
            a.push_back({n - 1, j});
        } else {
            a.push_back({n - 1, j});
            a.push_back({n - 2, j});
        }
    }

    for (int i = n - 1; i >= 2; i--) {
        if ((n - 1 - i) % 2 == 0) {
            a.push_back({i, 1});
            a.push_back({i, 0});
        } else {
            a.push_back({i, 0});
            a.push_back({i, 1});
        }
    }

    return a;
}

int main() {
    cin.tie(nullptr)->sync_with_stdio(false);

    int T;
    cin >> T;

    while (T--) {
        int n, m, rs, cs;
        cin >> n >> m >> rs >> cs;
        --rs;
        --cs;

        int N = n * m;

        auto id = [&](int x, int y) {
            return x * m + y;
        };

        auto getDir = [&](int u, int v) {
            int x1 = u / m, y1 = u % m;
            int x2 = v / m, y2 = v % m;

            if (x2 == x1 - 1) return 'U';

```

```

        if (x2 == x1 + 1) return 'D';
        if (y2 == y1 - 1) return 'L';
        return 'R';
    };

    deque<int> body;
    vector<int> occ(N);

    int start = id(rs, cs);
    body.push_front(start);
    occ[start] = 1;

    bool even = (n % 2 == 0 || m % 2 == 0);

    vector<int> ord;
    vector<int> pos(N, -1);

    int cur = start;

    if (even) {
        auto p = evenCycle(n, m);
        for (auto [x, y] : p) {
            ord.push_back(id(x, y));
        }
        for (int i = 0; i < N; i++) {
            pos[ord[i]] = i;
        }
    } else {
        auto p = oddBase(n, m);
        for (auto [x, y] : p) {
            ord.push_back(id(x, y));
        }
        for (int i = 0; i < int(ord.size()); i++) {
            pos[ord[i]] = i;
        }
    }

    int ptr = even ? pos[start] : -1;

    auto moveTo = [&](int v, bool eat) {
        if (!eat) {
            occ[body.back()] = 0;
            body.pop_back();
        }
        body.push_front(v);
        occ[v] = 1;
        cur = v;
    };

    for (int step = 0; step < N - 1; step++) {
        int r, c;
        cin >> r >> c;
        --r;
    }

```

```

--c;

int target = id(r, c);
string ans;

while (cur != target) {
    int nxt;

    if (even) {
        ptr = (ptr + 1) % N;
        nxt = ord[ptr];
    } else {
        int S = id(1, 0);
        int A = id(0, 0);
        int B = id(1, 1);
        int C = id(0, 1);

        if (cur == S) {
            if (target == A) {
                nxt = A;
            } else if (target == B) {
                nxt = B;
            } else {
                int tail = body.back();
                if (!occ[A] || tail == A) {
                    nxt = A;
                } else {
                    nxt = B;
                }
            }
        } else if (cur == B) {
            nxt = C;
        } else {
            nxt = ord[(pos[cur] + 1) % int(ord.size())];
        }
    }

    bool eat = (nxt == target);
    ans += getDir(cur, nxt);
    moveTo(nxt, eat);
}

cout << ans << '\n';
cout.flush();
}

return 0;
}

```

5. 总结与启示

难度评分

预估难度：

2000 ~ 2200

如果只考虑偶数行或偶数列，难度大约 1600；难点主要在奇数 × 奇数时的分支伪环构造。

核心启示

贪吃蛇类构造题的关键不是每次找最短路，而是维护一个永远不会把自己困死的全局不变量。

同类习题推荐

1. AtCoder ABC276 E - Round Trip
练习网格中的环与路径结构。
2. CSES - Hamiltonian Flights
练习“经过所有点一次”的哈密顿路径建模。
3. AtCoder ABC317 C - Remembering the Days
练习小规模图上的路径构造与状态分析。